



ESCOLA SENAI "A. JACOB LAFER"
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

RAFAELA OLIVEIRA

Atividade de Pesquisa: Mergulho nos Fundamentos de
Testes de Software

SANTO ANDRÉ

2026

RAFAELA OLIVEIRA

**Atividade de Pesquisa: Mergulho nos Fundamentos de
Testes de Software**

Trabalho apresentada ao curso Técnico em Desenvolvimento de sistemas da escola SENAI A. Jacob Lafer, como requisito parcial à obtenção do título de Técnico em Desenvolvimento de sistemas. Área de concentração: Teste de Software.

Orientador: Prof. André Luis Denani.

SANTO ANDRÉ

2026

RESUMO

O presente trabalho analisa o incidente tecnológico ocorrido na Knight Capital Group em 1º de agosto de 2012, amplamente reconhecido como um dos maiores desastres da história dos mercados financeiros. O estudo descreve, sob a ótica da engenharia de software e da gestão de qualidade, as causas técnicas do colapso, que resultou em perdas diretas estimadas entre US\$ 440 e 460 milhões em aproximadamente 45 minutos. A investigação identifica que a origem do problema esteve associada a falhas de implantação, manutenção inadequada de código legado e ausência de controles de risco adequados, evidenciando a importância de práticas robustas de teste de integração, de sistema e de aceitação.

A partir desse caso, o trabalho estabelece relações com conceitos fundamentais de requisitos funcionais e não funcionais, utilizando o aplicativo TikTok como exemplo prático para ilustrar como diferentes tipos de requisitos orientam o desenvolvimento e a avaliação de sistemas. Por fim, discute-se o papel dos níveis de teste, com ênfase para o teste unitário, o teste de integração e o teste de regressão, demonstrando que a combinação adequada dessas práticas é essencial para prevenir falhas sistêmicas e garantir a qualidade e a segurança de soluções de software.

Palavras-chave: Knight Capital. Teste de integração. Requisitos de software. Qualidade de software. Engenharia de software.

ABSTRACT

This paper analyzes the technological incident that occurred at Knight Capital Group on August 1, 2012, widely recognized as one of the largest disasters in financial market history. The study describes, from the perspective of software engineering and quality management, the technical causes of the collapse, which resulted in direct losses estimated between USD 440 and 460 million in approximately 45 minutes. The investigation identifies that the origin of the problem was associated with deployment failures, inadequate maintenance of legacy code, and lack of appropriate risk controls, highlighting the importance of robust practices in integration testing, system testing, and acceptance testing.

Based on this case, the paper establishes connections with fundamental concepts of functional and non-functional requirements, using the TikTok application as a practical example to illustrate how different types of requirements guide the development and evaluation of systems. Finally, the role of testing levels is discussed, with emphasis on unit testing, integration testing, and regression testing, demonstrating that the adequate combination of these practices is essential to prevent systemic failures and ensure the quality and safety of software solutions.

Keywords: Knight Capital. Integration testing. Software requirements. Software quality. Software engineering.

SUMÁRIO

Sumário

1 INTRODUÇÃO	6
2 O CUSTO DO ERRO: O CASO KNIGHT CAPITAL	7
2.1 Descrição técnica do incidente	7
2.2 Impacto financeiro e institucional	8
2.3 Análise dos níveis de teste	9
3 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS NA PRÁTICA: TIKTOK	10
3.1 Requisitos funcionais	10
3.2 Requisitos não funcionais	10
3.3 Cenário de teste de estresse	11
4 EXPLORANDO OS NÍVEIS DE TESTE	12
4.1 Teste unitário versus teste de integração	12
4.2 Teste de regressão	13
5 CONSIDERAÇÕES FINAIS	14
REFERÊNCIAS	15

1 INTRODUÇÃO

A qualidade de software e a confiabilidade dos sistemas tornaram-se aspectos críticos em um contexto em que organizações dependem cada vez mais de soluções automatizadas para operações de alta complexidade e grande impacto financeiro. Falhas de software em ambientes de missão crítica podem gerar prejuízos expressivos, danos reputacionais e até a extinção de empresas consolidadas. Nesse cenário, o incidente ocorrido na Knight Capital Group, em 1º de agosto de 2012, configura-se como um dos casos mais emblemáticos de falha tecnológica na área financeira.

O presente trabalho tem como objetivo analisar o caso Knight Capital sob a ótica da engenharia de software, identificando as causas técnicas do incidente, os níveis de teste que poderiam ter prevenido ou mitigado o problema e as lições aplicáveis ao desenvolvimento de sistemas em geral. Para tanto, descreve-se o contexto e a sequência de eventos que levaram ao colapso operacional da empresa, destacando-se aspectos relacionados à arquitetura do sistema SMARS, à gestão de código legado e à ausência de mecanismos de controle de risco.

A partir dessa análise, o trabalho estabelece relações entre o caso estudado e conceitos fundamentais de requisitos funcionais e não funcionais, utilizando o aplicativo TikTok como exemplo prático. Em seguida, discutem-se os níveis de teste de software, com ênfase para o teste unitário, o teste de integração e o teste de regressão, demonstrando que a combinação adequada dessas práticas é essencial para prevenir falhas sistêmicas e garantir a qualidade de soluções de software.

2 O CUSTO DO ERRO: O CASO KNIGHT CAPITAL

2.1 Descrição técnica do incidente

A Knight Capital Group era, até 2012, uma das maiores empresas de execução de ordens (*market maker*) do mercado acionário norte-americano, sendo responsável por uma parcela significativa do volume diário negociado nas bolsas de Nova York. Em 1º de agosto de 2012, a empresa se preparava para participar do *Retail Liquidity Program* (RLP), uma nova funcionalidade da *New York Stock Exchange* (NYSE) que oferecia melhores preços a investidores de varejo.

Para viabilizar essa participação, a Knight precisou atualizar o SMARS (*Share Monitoring and Routing System*), sistema responsável por rotear ordens de compra e venda para as bolsas. O problema técnico, contudo, teve origem em decisões de arquitetura e manutenção de código tomadas anos antes. Segundo análises posteriores do incidente, em meados dos anos 2000, um trecho de código foi realocado para um ponto anterior na sequência do programa, tornando obsoleta uma função interna chamada “Power Peg”, originalmente usada apenas em testes internos e em estratégias já descontinuadas. Esse código obsoleto, no entanto, nunca foi removido do ambiente de produção e permaneceu “dormente” em um dos servidores, reaproveitando um sinalizador (*flag*) que, em vez de ser eliminado, passou a controlar a nova funcionalidade do RLP.

Em julho de 2012, ao implantar o novo código do RLP, os técnicos da Knight realizaram o processo manualmente em oito servidores de produção. Um desses servidores não recebeu a atualização corretamente e manteve o código legado do Power Peg ativo. Na abertura do pregão, às 9h30 de 1º de agosto, as ordens marcadas para o RLP ativaram, nesse servidor específico, a rotina obsoleta, que passou a comprar e vender ações repetidamente sem reconhecer que as ordens já haviam sido executadas, pois a lógica de decremento da quantidade restante da ordem-mãe não existia mais nesse trecho de código.

O resultado foi um *looping* de ordens não intencionais: em aproximadamente 45 minutos, o sistema gerou cerca de 4 milhões de execuções, envolvendo aproximadamente 397 milhões de ações em mais de 140 empresas listadas, acumulando cerca de US\$ 7 bilhões em posições indesejadas. Não havia, na época, um mecanismo automático de interrupção (*kill switch*) que impedisse esse volume de ordens erráticas, e a intervenção manual dos engenheiros só ocorreu quando o estrago já havia sido feito. As perdas financeiras diretas foram estimadas entre US\$ 440 e 460 milhões, o que levou a uma queda acentuada no valor de mercado da empresa e à necessidade de captação emergencial de capital.

2.2 Impacto financeiro e institucional

O prejuízo direto da falha foi estimado entre US\$ 440 e 460 milhões, valor superior ao lucro anual da empresa. Nos dois dias seguintes ao incidente, as ações da Knight Capital registraram queda entre 75% e 80%, situação que colocou a companhia à beira da falência. A empresa só sobreviveu graças a um aporte emergencial de aproximadamente US\$ 400 milhões, proveniente de um consórcio de investidores, e, em 2013, foi incorporada pela concorrente Getco, dando origem à KCG Holdings — encerrando, na prática, a existência independente da Knight Capital, uma empresa que contava com 17 anos de operação.

Além do prejuízo financeiro direto, a *Securities and Exchange Commission* (SEC), órgão regulador do mercado de capitais dos Estados Unidos, aplicou à empresa, em 2013, uma multa de US\$ 12 milhões por violação da *Market Access Rule*. A autarquia concluiu que a Knight não dispunha de controles de risco adequados para impedir o envio descontrolado de ordens ao mercado, caracterizando falha grave na governança de acesso direto às bolsas (*direct market access*) e na gestão de riscos operacionais.

2.3 Análise dos níveis de teste

A falha da Knight Capital poderia ter sido encontrada, principalmente, no **nível de teste de integração**, com contribuição complementar do **teste de sistema** e do **teste de aceitação**.

Teste de integração

A causa imediata do desastre foi um erro de implantação: apenas sete dos oito servidores do SMARS receberam a nova versão do código, enquanto um manteve o código legado do *Power Peg* ativo. O teste de integração é o nível responsável por validar a interação entre componentes e a consistência do ambiente como um todo, neste caso, o cluster de servidores de produção.

Um teste de integração adequado teria verificado a consistência da versão do código em todos os servidores do cluster, exercitado o sistema com ordens de entrada idênticas em todos os nós e comparado os comportamentos, evidenciando a divergência de configuração antes da abertura do pregão e impedindo que um servidor desatualizado coexistisse com os demais em produção. Portanto, a falha é, essencialmente, uma falha de integração e implantação, que um teste de integração bem projetado poderia ter detectado.

Teste de sistema

Ao simular o fluxo real de ordens do RLP em um ambiente equivalente à produção (réplica ou *staging*), exercitando o sistema de ponta a ponta, o teste de sistema provavelmente exporia o comportamento anômalo do servidor desatualizado diante de ordens reais, funcionando como última barreira antes do *deploy* definitivo.

Teste de aceitação

Sob a ótica de negócio e gestão de risco, os critérios de aceitação deveriam exigir mecanismos de segurança, como limites de posição e *kill switch* automático. A ausência desses controles indica que os requisitos não funcionais de segurança operacional não foram adequadamente validados antes da liberação do sistema.

Conclui-se, portanto, que a falha da Knight Capital foi, sobretudo, uma falha de teste de integração — a validação de que o ambiente de produção, como um todo, refletia a versão de código pretendida —, agravada pela ausência de testes de aceitação relacionados a controles de risco. Isso demonstra, na prática, porque nenhum nível de teste é suficiente isoladamente.

3 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS NA PRÁTICA: TIKTOK

3.1 Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, as ações e funcionalidades diretamente visíveis e utilizadas pelo usuário. No caso do TikTok, podem-se destacar os seguintes requisitos funcionais:

- Ao deslizar a tela para cima (*swipe up*), o aplicativo deve interromper a reprodução do vídeo atual e reproduzir automaticamente o próximo vídeo do feed “Para Você”.

- Ao tocar duas vezes na tela (*double tap*) ou no ícone de coração, o vídeo em exibição deve ser curtido, e o contador de curtidas exibido deve ser atualizado imediatamente.

- Durante a gravação de um vídeo, o usuário deve conseguir aplicar filtros visuais, efeitos sonoros e adicionar legendas ou texto antes de publicar o conteúdo.

Esses requisitos expressam comportamentos esperados do sistema a partir de interações específicas do usuário, constituindo a base para o desenvolvimento das funcionalidades centrais da aplicação.

3.2 Requisitos não funcionais

Os requisitos não funcionais descrevem como o sistema deve se comportar, isto é, atributos de qualidade como desempenho, segurança e escalabilidade, que não são visíveis como uma função específica, mas que impactam diretamente a experiência do usuário. No caso do TikTok, podem-se destacar os seguintes requisitos não funcionais:

- O próximo vídeo do feed deve iniciar a reprodução em menos de 1 a 2 segundos após o usuário deslizar a tela, garantindo fluidez na navegação (desempenho).

- O aplicativo deve suportar picos de dezenas de milhões de usuários acessando simultaneamente, sem degradação perceptível de desempenho (escalabilidade e disponibilidade).

- Os dados pessoais, vídeos e mensagens dos usuários devem ser armazenados de forma criptografada e em conformidade com legislações de proteção de dados, como a Lei Geral de Proteção de Dados (LGPD) e o *General Data Protection Regulation* (GDPR) (segurança e privacidade).

Esses requisitos estabelecem restrições e critérios de qualidade para o sistema, influenciando decisões de arquitetura, infraestrutura e implementação, ainda que não correspondam a funcionalidades diretamente observáveis pelo usuário final.

3.3 Cenário de teste de estresse

Um cenário adequado para testar o limite do TikTok seria simular um evento de viralização repentina: um único vídeo passa a ser acessado, curtido e comentado por milhões de usuários simultaneamente em um curto intervalo de tempo (por exemplo, o número de requisições por segundo crescendo de 10 mil para 5 milhões em poucos minutos). Nesse teste de estresse, seriam observados: o tempo de resposta do carregamento do feed sob carga extrema; a taxa de erros nas requisições de curta e comentário; a consistência do contador de visualizações e curtidas exibido a diferentes usuários ao mesmo tempo; e o comportamento do sistema de recomendação (algoritmo do "Para Você") diante do pico, verificando se o sistema degrada de forma controlada (por exemplo, atrasando atualizações não críticas) em vez de ficar indisponível.

4 EXPLORANDO OS NÍVEIS DE TESTE

4.1 Teste unitário versus teste de integração

O teste unitário verifica isoladamente a menor unidade testável de um sistema, como uma função, um método ou uma pequena classe, assumindo que tudo ao redor dela funciona corretamente. Já o teste de integração verifica se essas unidades, quando combinadas e implantadas em conjunto, em diferentes módulos, serviços ou até servidores, continuam se comportando corretamente.

Não é possível realizar apenas um dos dois porque cada um cobre um tipo de falha que o outro não enxerga. O caso Knight Capital, discutido no Capítulo 1, ilustra esse ponto com precisão. É plausível que, isoladamente, cada trecho de código da Knight passasse em seus testes unitários. A lógica de roteamento de ordens, considerada função por função, poderia estar tecnicamente correta. O problema real só existia quando se observava o sistema como um todo, já implantado em produção: sete servidores tinham uma versão do código, e um oitavo tinha outra, algo que nenhum teste unitário jamais revelaria, pois o teste unitário não avalia o ambiente de implantação nem a consistência entre múltiplas instâncias do sistema. Somente um teste de integração, validando o comportamento do cluster completo diante de ordens reais do RLP, seria capaz de expor essa inconsistência. Por isso, um nível de teste não substitui o outro: o teste unitário garante a corretude de cada peça, e o teste de integração garante que as peças, juntas, formam um sistema coerente.

4.2 Teste de regressão

O teste de regressão consiste em reexecutar testes sobre funcionalidades já existentes e previamente validadas, após qualquer alteração no sistema, com o objetivo de garantir que essa mudança não tenha introduzido efeitos colaterais indesejados em partes que já funcionavam.

Ao adicionar um novo botão de "Esqueci minha senha" a uma tela de login, é necessário testar novamente o botão de "Login", mesmo que ele não tenha sido alterado diretamente, porque a nova funcionalidade compartilha o mesmo contexto: a mesma tela, o mesmo formulário, possivelmente as mesmas validações de campo e as mesmas chamadas ao servidor de autenticação. Uma mudança aparentemente isolada pode, sem intenção, quebrar um fluxo vizinho. Por exemplo, uma reorganização no layout pode fazer o botão de login perder o evento de clique, ou uma alteração na validação de campos pode passar a bloquear indevidamente o envio do formulário de login original.

Esse é, essencialmente, o mesmo tipo de falha que ocorreu na Knight Capital, guardadas as devidas proporções de escala e criticidade. Em 2005, uma alteração aparentemente pontual, ao mover um trecho de código para um ponto anterior na sequência do programa, tornou uma função já existente, o *Power Peg*, obsoleta e potencialmente perigosa, sem que ninguém percebesse o efeito colateral na época. Esse código comprometido permaneceu latente por sete anos até ser reativado por uma mudança completamente diferente, a implantação do RLP, causando o colapso da empresa. Se práticas de teste de regressão tivessem sido aplicadas de forma consistente a cada alteração no SMARS ao longo dos anos, a existência de um código obsoleto e mal desativado dentro do sistema provavelmente teria sido identificada muito antes de causar prejuízo. O teste de regressão existe justamente para evitar que um "Knight Capital em miniatura" aconteça em qualquer sistema, por menor que seja a alteração realizada.

5 CONSIDERAÇÕES FINAIS

A análise do caso Knight Capital demonstra, de forma contundente, que os níveis de teste estudados no Capítulo 1 não são etapas burocráticas, mas camadas de proteção complementares: cada uma delas é capaz de capturar um tipo específico de falha que as demais não alcançam. A empresa perdeu, em 45 minutos, o equivalente a 17 anos de construção de reputação e negócios, principalmente por falhas de integração e de controles de aceitação de risco — não por um erro trivial de programação.

A análise do TikTok, por sua vez, evidenciou que requisitos funcionais e não funcionais são igualmente importantes: de nada adianta um aplicativo com funcionalidades ricas (curtir, comentar, gravar vídeos) se ele não é capaz de sustentar o desempenho e a segurança exigidos por uma base de usuários massiva — e é justamente essa dimensão não funcional que testes de estresse ajudam a validar.

Por fim, ao revisitar o caso Knight Capital para explicar teste unitário, teste de integração e teste de regressão, ficou evidente que esses conceitos não são exclusivos de ambientes acadêmicos: eles existem porque sistemas reais, complexos e em constante evolução, estão sujeitos a efeitos colaterais imprevisíveis. Testar cada unidade isoladamente, testar a integração entre componentes e reexecutar testes de regressão a cada mudança são práticas que, se negligenciadas, podem custar — como no caso estudado — centenas de milhões de dólares em poucos minutos.

REFERÊNCIAS

BBC. High-frequency trading and the \$440m mistake. BBC News, 10 ago. 2012. Disponível em: <https://www.bbc.com/news/magazine-19214294>. Acesso em: 17 set. 2026.

BUSINESS INSIDER. Knight Capital is facing a \$440 million loss after yesterday's trading glitch. Business Insider, 2 ago. 2012. Disponível em: <https://www.businessinsider.com/knight-capital-is-facing-a-440-million-loss-after-yesterdays-trading-glitch-2012-8>. Acesso em: 17 set. 2026.

CIO. Software testing lessons learned from Knight Capital fiasco. CIO, 14 ago. 2012. Disponível em: <https://www.cio.com/article/286790/software-testing-lessons-learned-from-knight-capital-fiasco.html>. Acesso em: 17 set. 2026.

EXAME. Knight Capital perde US\$ 440 milhões por falha em robô. Exame, [s. d.]. Disponível em: <https://exame.com/invest/mercados/knight-capital-perde-us-440-milhoes-por-falha-em-robo/>. Acesso em: 17 set. 2026.

FAILUREPEDIA. Knight Capital: why it failed. Failurepedia, 22 jul. 2026. Disponível em: <https://www.failurepedia.com/cases/knight-capital>. Acesso em: 17 set. 2026.

MARKET HISTORIES. The Knight Capital meltdown: how 45 minutes of bad code cost \$440 million (2012). Market Histories, 17 maio 2026. Disponível em: <https://www.markethistories.com/en/the-knight-capital-meltdown-how-45-minutes-of-bad-code-cost-440-million-2012>. Acesso em: 17 set. 2026.

PERPLEXITY AI. Perplexity. Assistente de inteligência artificial. 2026. Disponível em: <https://www.perplexity.ai>. Acesso em: 17 set. 2026.

PRMIA. The Knight Capital algorithmic trading failure. PRMIA Case Study. Disponível em: <https://prmia.org/common/Uploaded%20files/eAI/PRMIA%20Case%20study%20-%20Knight%20Trading.pdf>. Acesso em: 17 set. 2026.

REUTERS. Knight Capital posts \$389.9 million loss on trading glitch. Reuters, 17 out. 2012. Disponível em: <https://www.reuters.com/article/business/knight-capital-posts-3899-million-loss-on-trading-glitch-idUSBRE89G0HJ/>. Acesso em: 17 set. 2026.

REUTERS. Knight trading loss shows cracks in equity markets. Reuters, 3 ago. 2012. Disponível em: <https://www.reuters.com/article/business/knight-trading-loss-shows-cracks-in-equity-markets-idUSBRE87203X/>. Acesso em: 17 set. 2026.

TABBA, Bishr. The rise and fall of Knight Capital — buy high, sell low. Rinse and repeat. Medium, 5 ago. 2018. Disponível em: <https://medium.com/dataseries/the-rise-and-fall-of-knight-capital-buy-high-sell-low-rinse-and-repeat-ae17fae780f6>. Acesso em: 17 set. 2026.

THE NEW YORK TIMES. Knight Capital says trading glitch cost it \$440 million. DealBook, 2 ago. 2012. Disponível em: <https://dealbook.nytimes.com/2012/08/02/knight-capital-says-trading-mishap-cost-it-440-million/>. Acesso em: 17 set. 2026.

TRADERS MAGAZINE. The rise and fall of Knight Capital — buy high, sell low. Rinse and repeat. Traders Magazine, 10 set. 2019. Disponível em: <https://www.tradersmagazine.com/departments/brokerage/the-rise-and-fall-of-knight-capital-buy-high-sell-low-rinse-and-repeat-2/>. Acesso em: 17 set. 2026.